

Resumen express - Arquitectura de Computadores

Clase del martes 14 de abril - Relación con el Sistema Operativo

Fuente: video abril-14.mp4. Enfoque: repaso rápido para trabajo práctico.

Idea central de la clase

La clase conecta arquitectura 80x86/MASM con el Sistema Operativo: procesos, concurrencia, interrupciones y uso de servicios del SO desde ensamblador. Al final se vuelve al programa de suma de cubos para mostrar un mensaje en consola usando servicios de Windows.

1. Mapa rápido de la clase

- Diferencia entre programa y proceso.
- Procesamiento paralelo vs procesamiento concurrente.
- Estados de un proceso: new, ready, running, waiting, terminated.
- Tres razones para activar una interrupción: servicios del SO, excepciones y fin del quantum.
- Mecanismo de interrupción en 80x86: drivers/handlers de interrupción y la instrucción int <nm>.
- Aplicación práctica en MASM: uso de kernel32.dll, GetStdHandle y WriteFile para imprimir un mensaje en consola.

2. Programas y procesos

Un programa es código almacenado: instrucciones y datos que podrían ejecutarse. Un proceso es ese programa mientras está en ejecución. Cuando un programa corre, el contenido de registros y RAM cambia constantemente.

La computadora suele tener muchos procesos activos. Algunos realmente corren en paralelo si hay varios núcleos, y otros corren de forma concurrente, alternándose muy rápido en el mismo procesador.

3. Procesamiento paralelo vs concurrente

Concepto	Qué significa	Ejemplo
Paralelo	Dos o más procesos avanzan literalmente al mismo tiempo porque existen varios núcleos/CPU.	Un proceso en Core 1 y otro en Core 2.
Concurrente	Varios procesos parecen avanzar al mismo tiempo, pero se turnan el procesador en intervalos muy pequeños.	Mientras escribes en Word, el sistema atiende teclado, pantalla, antivirus, audio, red, etc.

Ejemplo de la diapositiva: una persona escribe 4 caracteres por segundo; hay 0,25 s entre teclazos. Con un procesador de 2 GHz y 8 ciclos por instrucción, hay 250 000 000 instrucciones por segundo. Ojo: si la

pregunta se interpreta estrictamente como “entre dos teclazos” usando 0,25 s, el cálculo da 62 500 000 instrucciones. La diapositiva anotó 250 000 000, que corresponde al cálculo por segundo.

4. Estados de un proceso

El diagrama usado en clase muestra los estados típicos de un proceso y las transiciones que el SO administra:

Estado	Qué representa	Transiciones importantes
new	Proceso recién creado.	new -> ready cuando es admitido.
ready	Está listo para correr, pero espera CPU.	ready -> running por scheduler dispatch.
running	Está ejecutando instrucciones en CPU.	running -> ready por interrupción; running -> waiting por I/O; running -> terminated por exit.
waiting	Está bloqueado esperando I/O o evento.	waiting -> ready cuando termina el I/O o evento.
terminated	Proceso finalizado.	Llega aquí cuando termina normalmente o por excepción grave.

Cómo pensarlo en el trabajo

El SO decide quién usa el procesador. Para poder cambiar de proceso, detener uno, despertar otro o atender una petición, necesita romper la ejecución secuencial normal. Ese mecanismo se llama interrupción.

5. Interrupciones: por qué existen

Una interrupción es una alteración controlada de la secuencia normal de ejecución. La CPU deja temporalmente lo que estaba haciendo, guarda el contexto necesario, ejecuta una rutina del SO y luego continúa o cambia a otro proceso.

En clase se indicaron tres razones principales para activar el mecanismo de interrupción:

1. Solicitud de servicios del Sistema Operativo: abrir/cerrar archivos, leer/escribir archivos, operaciones con periféricos, consultar la hora del sistema, etc. En 80x86, para este caso aparece la idea de la instrucción int.
2. Excepciones: instrucción ilegal, división entre cero, acceso ilegal a RAM, entre otras. En estos casos, la Unidad de Control detecta el problema y puede terminar el proceso.
3. Fin del quantum: cada proceso recibe una porción de tiempo. Cuando el temporizador llega a cero, se interrumpe el proceso actual y vuelve a ready para que el planificador le dé oportunidad a otro proceso.

6. Mecanismo de interrupción en 80x86

Cada arquitectura define su propio mecanismo de interrupción. En 80x86, cada situación diferente necesita una rutina diferente para atenderla. Esas rutinas pertenecen al SO y en la clase se les llamó interrupt drivers.

Para servicios del SO, la forma conceptual mostrada fue:

```
int <inm>
```

El operando inmediato <inm> identifica la posición del vector de interrupciones del driver que se quiere activar. Para interrupciones externas o excepciones, no necesariamente lo pide el programa: la Unidad de Control identifica qué rutina debe invocar.

7. Aplicación en MASM: imprimir mensaje desde ensamblador

La clase retoma el programa de suma de cubos y agrega comunicación con el SO para escribir un mensaje en consola. En vez de implementar directamente una interrupción cruda, el programa llama funciones de Windows desde kernel32.dll.

Declaraciones importantes

```
includelib \Windows\System32\kernel32.dll
ExitProcess proto

extrn GetStdHandle : proc
extrn WriteFile : proc

.data
    limit    equ 5
    n        dw 1
    SumaCubos dd 2
    CuadradoSuma dd 3

    mensaje1 db "Digite el limite de la suma de los cubos: ",0Dh,0Ah
    msg1Len   equ $-mensaje1
    bytesEscritos dq 0
```

Puntos clave:

- includelib enlaza la biblioteca dinámica de Windows. El nombre correcto es kernel32.dll.
- extrn declara procedimientos externos para que el linker pueda resolverlos.
- mensaje1 es una hilera de bytes; 0Dh y 0Ah son retorno de carro y salto de línea.
- msg1Len equ \$-mensaje1 calcula en ensamblado la longitud del mensaje.
- bytesEscritos guarda cuántos bytes fueron escritos por WriteFile.

Procedimiento getSumLimit visto en clase

```
getSumLimit PROC
    mov RBP,RSP
; StackFrame:
;   RSP : Return Address
;   RSP+8: Returned Value

    sub RSP,32          ; Shadow space de Windows x64
    mov RCX,-11         ; STD_OUTPUT_HANDLE para consola
```

```

call GetStdHandle
mov RBX,RAX          ; RAX trae el handle

; WriteFile(handle, buffer, length, bytesWritten, overlapped)
mov RCX,RBX          ; 1er argumento: handle de consola
lea RDX,mensaje1      ; 2do argumento: dirección del mensaje
mov R8,msg1Len        ; 3er argumento: longitud
lea R9,bytesEscritos   ; 4to argumento: puntero a contador
mov qword ptr [RSP+32],0 ; 5to argumento: NULL

call WriteFile
add RSP,32            ; Restaurar RSP
mov dword PTR [RSP+8],limit; Valor de retorno simulado
ret
getSumLimit ENDP

```

La convención de llamada de Windows x64 pasa los primeros cuatro argumentos en RCX, RDX, R8 y R9. Si hay un quinto argumento, se coloca en la pila. Además, el caller reserva 32 bytes de shadow space antes de llamar.

8. Errores típicos que conviene revisar

- No escribir kernel132.dll: debe ser kernel32.dll.
- No confundir -11: en Windows, -11 corresponde a STD_OUTPUT_HANDLE; para entrada estándar suele usarse -10.
- Si se usa WriteFile para mostrar texto, el handle debe ser de salida estándar, no de entrada.
- Después de sub RSP,32 debe aparecer add RSP,32 antes de retornar, para restaurar la pila.
- La quinta posición de argumento para WriteFile se coloca como NULL: mov qword ptr [RSP+32],0.
- Antes de acceder al valor retornado en [RSP+8], hay que asegurarse de que RSP volvió a apuntar al stack frame original.
- En x64, cuidar la alineación de pila: las llamadas a funciones de Windows esperan la pila alineada correctamente.

9. Conexión con el programa de suma de cubos

El programa principal venía de la clase anterior: calcula la suma de cubos en R8d, calcula la fórmula del cuadrado de la suma en R9d, compara ambos resultados y llama a anunciarIgualdad o anunciarDesigualdad. En esta clase, el foco cambia a cómo pedir/mostrar información usando servicios del SO.

```

sub RSP, 8
call getSumLimit
pop RCX
mov n, CX

xor R8,R8
call sumCubes

```

```

xor R9,R9
call formulaCuadradoSuma

cmp R8,R9
jne elseComparar
    call anunciarIgualdad
    jmp endIfResultadosIguales
elseComparar:
    call anunciarDesigualdad
endIfResultadosIguales:
call ExitProcess

```

La idea práctica es que getSumLimit deja el límite de la suma como valor retornado. En el código mostrado todavía se simula con limit equ 5, pero ya se agrega el mensaje en consola como primer paso hacia entrada/salida real.

10. Chuleta rápida para consultas

Tema	Recordatorio
Proceso	Programa en ejecución; modifica registros y RAM.
Concurrente	Se alternan procesos rápidamente en el CPU.
Paralelo	Procesos corren al mismo tiempo en núcleos distintos.
Ready	Listo para correr, esperando CPU.
Running	Ejecutando en CPU.
Waiting	Esperando I/O o evento.
Interrupción	Mecanismo para romper secuencia normal y atender SO/excepción/temporizador.
Quantum	Porción de tiempo asignada a un proceso.
int <inm>	Activa un driver/rutina de interrupción identificado por el inmediato.
Windows x64 args	RCX, RDX, R8, R9; luego pila.
Shadow space	32 bytes reservados antes de llamar funciones Windows x64.
WriteFile	Función para escribir bytes en un handle, como la consola.

Lo más probable que te pregunten en el trabajo

Explicar por qué el SO necesita interrupciones, interpretar el diagrama de estados de un proceso, diferenciar concurrencia/paralelismo, y entender el paso de argumentos a GetStdHandle/WriteFile en MASM x64.

Resumen preparado para repaso rápido. Revisa el código final del proyecto si el profesor corrigió nombres o comentarios durante la clase.